
HavenWorld

Phase 0 & Phase 1 Complete Build Guide

From Blank Repo to Small Group Alpha Testing

Author: Daniel Boone

Project: dboone323/HavenWorld | havenworld.me

Date: September 17, 2026

Document Type: Technical Build Guide — Confidential / Internal

Version: 1.0

Table of Contents

1. Overview & Repo Audit

- 1.1 Document Purpose
- 1.2 Repository Audit — dboone323/HavenWorld
- 1.3 Competitive Context
- 1.4 Recommended Tech Stack
- 1.5 Recommended Monorepo Structure

2. Phase 0 — Everything to Set Up First

- 2.1 Local Development Environment
- 2.2 Repository Setup & Organization
- 2.3 Cloud Services Setup
- 2.4 Environment Variables & Secrets
- 2.5 CI/CD Pipeline (GitHub Actions)
- 2.6 Local Development Database (Docker)
- 2.7 Database Schema Foundation (Prisma)

- 2.8 Discord Server & Community Setup
- 2.9 Pre-Launch Landing Page & Dev Blog
- 2.10 Legal & Compliance
- 2.11 Phase 0 Completion Checklist

3. Phase 1 — Building the Game for Small Group Testing

- 3.1 Project Bootstrap
- 3.2 Authentication System
- 3.3 Real-Time Game Server (Socket.io)
- 3.4 Avatar System
- 3.5 World & Room System
- 3.6 Chat System
- 3.7 Moderation System
- 3.8 Friend System
- 3.9 Closed Alpha Infrastructure
- 3.10 Performance Targets & Testing
- 3.11 Phase 1 Alpha-Ready Gate Checklist

4. Phase 1 → Phase 2 Preview

5. Estimated Timeline

1. Overview & Repo Audit

1.1 Document Purpose

This is the complete, authoritative, step-by-step build guide for HavenWorld — a browser-based social virtual world developed by Daniel Boone under the GitHub repository [dboone323/HavenWorld](https://github.com/dboone323/HavenWorld). The document covers two sequential phases that must be completed in order before any testers are ever invited to the platform.

This guide covers the following phases in full:

- **Phase 0: Foundation** — Everything that must be set up BEFORE writing a single line of game code. Estimated duration: **1-2 weeks**. Covers local

tooling, cloud services, CI/CD, database schema, Discord community, legal compliance, and organizational scaffolding.

- **Phase 1: Core Loop Alpha** — The development steps required to produce a playable, stable game ready for **5-25 trusted testers**. Estimated duration: **6-10 weeks**. Covers authentication, real-time sockets, avatar system, room system, chat, moderation, friends, and alpha logistics.

CRITICAL: Read Before Starting

Phase 0 is non-negotiable. No game can be built sustainably on an unscaffolded codebase with no CI, no database schema, no secrets management, and no legal coverage. Skipping Phase 0 and jumping into game code is the single most common reason indie social world projects collapse before alpha. Complete every item in Phase 0 before writing any game logic.

1.2 Repository Audit — dboone323/HavenWorld

As of September 17, 2026, the HavenWorld repository at github.com/dboone323/HavenWorld was audited and found to be in the following state. This confirms the starting point for all work described in this guide.

Repository Item	Status	Action Required
License	GPL-3.0 (confirmed)	Review dual-license decision (Section 2.10)
Commit count	1 initial commit	None — clean slate confirmed
Game code	NONE — blank slate	Build per Phase 1 of this guide
Frontend	NONE	Scaffold per Phase 1, Step 2
Backend / Server	NONE	Scaffold per Phase 1, Step

Repository Item	Status	Action Required
		1
Database schema	NONE	Define in Phase 0, Section 2.7
.gitignore	Present (basic)	Expand to cover .env, node_modules, dist
README.md	Missing	Create in Phase 0, Step 8
havenworld.me domain	Registered (confirmed)	Connect to Cloudflare in Phase 0, Step 13
CI/CD pipelines	NONE	Create in Phase 0, Section 2.5
Cloud services	NONE provisioned	Provision all in Phase 0, Section 2.3

Key Finding

HavenWorld is a clean slate — this is a strength, not a weakness. There is no legacy technical debt to untangle. Everything built from this guide will be intentional, well-structured, and maintainable from Day 1. Phase 0 must be completed in full before Phase 1 begins.

1.3 Competitive Context

HavenWorld enters a market with well-established genre precedents. Understanding what made comparable social worlds succeed — and fail — is essential context for every architectural and design decision in this guide. The competitive analysis covers YoWorld, MegaPlanet, Club Penguin, and Habbo Hotel.

Title	Core Lesson	Application to HavenWorld
Habbo Hotel	20+ year run built on a solid WebSocket server and deeply layered moderation infrastructure.	Build moderation and real-time server architecture in Phase 0/1, not as an afterthought.

Title	Core Lesson	Application to HavenWorld
	Both must be designed before launch.	
Club Penguin	Tech was simple. The culture — events, identity, seasonality — was the actual product. Day-1 community beats Day-1 features.	Discord + dev blog + alpha community launched in Phase 0, before any game code.
MegaPlanet	Closest modern indie comparator. Started with basic rooms, chat, and avatar before adding jobs, fishing, and badges. Sequenced features correctly.	Phase 1 scope deliberately limited to: Avatar + Room + Chat. Economy comes in Phase 2.
YoWorld	Facebook-platform dependency caused near-death when platform policies changed. No owned distribution channel.	HavenWorld owns havenworld.me — never build on rented land for the primary client.

The universal lesson from all long-running social worlds: **nail the core loop first**. The core loop for HavenWorld Phase 1 is: **Log in → Customize Avatar → Enter a Room → Move Around → Chat with Others → Visit Personal Room**. Every feature in Phase 1 exists to make this loop feel alive, polished, and worth returning to. Everything else — economy, mini-games, events, monetization — is a Phase 2+ concern.

HavenWorld's Phase 1 goal: Make the core loop feel alive and genuinely fun for 5–25 trusted alpha testers. Day-30 retention exceeding 20% is the gate metric before any Phase 2 feature work begins.

1.4 Recommended Tech Stack

The following technology choices were made to optimize for: rapid development by a solo or small team, production-grade WebSocket performance suitable for a social world, zero or near-zero hosting cost during alpha, and long-term maintainability. Every choice is justified below.

Backend Stack

Component	Choice	Version	Reason
Runtime	Node.js	22 LTS	LTS release, excellent async/WebSocket performance, massive ecosystem
Framework	Express	5.x	Mature, minimal, huge ecosystem, well-understood by contributors
Real-time	Socket.io	4.x	Native rooms, automatic reconnection, ideal for social worlds — see note below
Auth	JWT + bcrypt + httpOnly cookies	—	Industry standard, secure, stateless, no server-side session storage required
ORM	Prisma	Latest	Type-safe queries, built-in migration tooling, excellent PostgreSQL support

Database Stack

Component	Choice	Cost (Alpha)	Reason
Primary DB	PostgreSQL 16	Free (Neon)	Relational model is perfect for users, items, rooms, economy, reports
Cache / Sessions	Redis 7	Free (Upstash)	Sub-millisecond reads, pub/sub for cross-server events, online user tracking
Hosted DB	Neon.tech	Free (0.5 GB)	Branching for dev/prod at zero extra cost;

Component	Choice	Cost (Alpha)	Reason
			serverless PostgreSQL
Hosted Redis	Upstash	Free (10K req/day)	Serverless Redis, no cold starts, generous free tier for alpha scale

Frontend Stack

Component	Choice	Reason
Game Engine	Phaser 3	Best HTML5 2D engine; WebGL+Canvas fallback; huge community; MIT license; free
UI Overlays	Vanilla HTML/CSS + Alpine.js	Chat, menus, inventory — keeps bundle minimal, no React overhead
Build Tool	Vite 6	Fast HMR, excellent Phaser integration, ES modules native
Sprite Format	TexturePacker JSON atlas	Efficient GPU texture atlases for layered avatar rendering

Infrastructure Stack

Component	Choice	Cost (Alpha)	Key Benefit
Game Server Host	Railway	\$5–20/mo	WebSocket-friendly; no cold starts; native GitHub deploys; zero DevOps
Static Frontend	Cloudflare Pages	Free	Global CDN; instant GitHub deploys; unlimited bandwidth
CDN / DNS / DDoS / SSL	Cloudflare (free)	Free	havenworld.me already registered; HTTP/3; Brotli; Bot

Component	Choice	Cost (Alpha)	Key Benefit
			Fight Mode
CI/CD	GitHub Actions	Free	2,000 min/mo on public repos; native GitHub integration
Error Tracking	Sentry (free)	Free	5K errors/mo; source maps; performance monitoring
Email	Resend (free)	Free	3,000 emails/mo; havenworld.me domain verification; modern SDK
Uptime Monitor	UptimeRobot (free)	Free	50 monitors; 5-minute checks; Discord webhook alerts

Why Socket.io Over Colyseus?

Colyseus is an excellent framework for

fast-paced competitive games

requiring tight state synchronization (e.g., shooters, racing games). HavenWorld is a social world — slower-paced, event-driven, room-based, with the primary value being presence and conversation, not reaction-time gameplay. Socket.io's native room abstraction maps perfectly to HavenWorld's room model, its reconnection handling is battle-tested, and its ecosystem is vastly larger. Colyseus can be added later as a secondary server specifically for fast-paced mini-games in Phase 3+.

1.5 Recommended Monorepo Structure

HavenWorld uses a **pnpm workspace monorepo** to co-locate the server and client in a single repository. This enables shared TypeScript types and Socket.io event

constants, atomic commits across the full stack, and unified CI/CD pipelines. The structure below must be scaffolded in Phase 0, Step 5 before any code is written.

```
HavenWorld/ └─ apps/ └─ server/ # Node.js + Express +
Socket.io game server └─ src/ └─ routes/ # REST
API (auth, users, rooms, items) └─ sockets/ # Socket.io
event handlers └─ services/ # Business logic (avatar, room,
chat) └─ middleware/ # Auth, rate limit, moderation └─
└─ prisma/ # Schema and migrations └─ package.json └─
client/ # Phaser 3 + Vite frontend └─ src/ └─
scenes/ # Phaser scenes (Boot, Lobby, Room, UI) └─
entities/ # Avatar, Furniture classes └─ ui/ # HTML
overlay components (chat, inventory) └─ assets/ # Sprites,
audio, tilemaps └─ package.json └─ packages/ └─ shared/
# Shared types, constants, Socket event names └─ .github/ └─ workflows/
# CI/CD pipelines └─ ROADMAP.md └─ CONTRIBUTING.md └─ SECURITY.md └─
package.json # Workspace root (pnpm workspaces)
```

PHASE 0 — FOUNDATION

|

ESTIMATED TIME: 1–2 WEEKS

🛑 STOP — Read This First

Complete ALL items in Phase 0 before writing a single line of game code. Every section below produces infrastructure, tooling, or organizational scaffolding that game code depends on. Starting Phase 1 without Phase 0 complete will result in rework, security holes, and a fragile codebase.

2. Phase 0 — Everything to Set Up First

2.1 Local Development Environment

Your local machine must be configured with the correct versions of all tools before initializing the repository structure. Mismatched Node versions are a leading cause of "works on my machine" CI failures.

Step 1 — Install Core Tools

- **Node.js 22 LTS** via `nvm` (Node Version Manager — never install Node globally via installer):

```
nvm install 22 && nvm use 22 && nvm alias default 22
```

- **pnpm** (monorepo package manager — faster and more disk-efficient than npm/yarn):

```
npm install -g pnpm
```

- **Git 2.40+** — verify with `git --version`
- **Docker Desktop** — runs PostgreSQL and Redis locally without any cloud dependency during development
- **VS Code** — recommended IDE; required extensions in Step 2

Step 2 — VS Code Extensions (install all)

Extension Name	ID	Purpose
ESLint	dbaeumer.vscode-eslint	Inline lint errors as you type
Prettier	esbenp.prettier-vscode	Format-on-save code formatting
Prisma	Prisma.prisma	Schema syntax highlighting and formatting
GitLens	eamodio.gitlens	Inline git blame, history, and branch management

Extension Name	ID	Purpose
Thunder Client	rangav.vscode-thunder-client	REST API testing inside VS Code
Docker	ms-azuretools.vscode-docker	Manage Docker containers from the sidebar
DotENV	mikestead.dotenv	.env file syntax highlighting

Step 3 — Configure Git Identity

```
git config --global user.name "Daniel Boone" git config --global user.email "dboone9@msn.com" git config --global core.autocrlf input git config --global pull.rebase true
```

Step 4 — SSH Key for GitHub

```
ssh-keygen -t ed25519 -C "dboone9@msn.com" # Add ~/.ssh/id_ed25519.pub to GitHub → Settings → SSH Keys ssh -T git@github.com # verify — should return: Hi dboone323!
```

2.2 Repository Setup & Organization

Step 5 — Initialize Monorepo Directory Structure

Run these commands from the repository root after cloning:

```
mkdir -p apps/server/src/{routes,sockets,services,middleware,prisma} mkdir -p apps/client/src/{scenes,entities,ui,assets/{sprites,audio,maps}} mkdir -p packages/shared/src mkdir -p .github/{workflows,ISSUE_TEMPLATE}
```

Step 6 — Root package.json (pnpm Workspaces)

```
{  "name": "havenworld",  "private": true,  "engines": { "node": ">=22.0.0", "pnpm": ">=9.0.0"},  "packageManager": "pnpm@9.0.0",  "workspaces": ["apps/*", "packages/*"],  "scripts": {    "dev": "pnpm --parallel --filter './apps/*' dev",    "build": "pnpm --filter './apps/*' build",    "test": "pnpm --filter './apps/*' test",    "lint": "pnpm --filter './apps/*' lint"  } }
```

Step 7 — Branch Strategy

Branch	Purpose	Protection Rules
main	Production-only; deploys directly to live servers	Require PR + all status checks pass; no direct push; require 1 review
develop	Integration branch; all	Require PR + CI pass;

Branch	Purpose	Protection Rules
	feature PRs merge here first	direct push allowed for solo dev
feature/*	Individual features (e.g., feature/avatar-system)	None — branch freely
hotfix/*	Emergency production fixes only	PR to main required; merge also to develop

In GitHub: **Settings** → **Branches** → **Add protection rule** for `main`. Enable: Require status checks, Require PR reviews before merging, Do not allow bypassing.

Step 8 — Essential Repository Files to Create

README.md must include: HavenWorld elevator pitch and value proposition; live URL badge (<https://havenworld.me>); tech stack badges (Node.js, Phaser 3, PostgreSQL, Socket.io); local setup instructions (clone → npm install → npm dev); links to ROADMAP.md, CONTRIBUTING.md, and Discord invite. This is the first thing any contributor or tester sees — make it excellent.

ROADMAP.md: Use GitHub checkboxes (- []) for interactive rendering. Organize into four sections: Phase 0 Foundation, Phase 1 Core Loop Alpha, Phase 2 Economy and Mini-games, Phase 3 Public Beta. Each item is a discrete deliverable, not a vague goal.

CONTRIBUTING.md: Fork setup instructions, branch naming conventions (feature/, hotfix/), PR template requirements, ESLint/Prettier configuration reference, code of conduct, and guidance on the `good first issue` label for new contributors.

SECURITY.md: Report vulnerabilities to security@havenworld.me (never to public issues); 72-hour acknowledgement SLA; note on GPL-3.0 dual-license implications for security researchers.

Step 9 — GitHub Issue & PR Templates

Create `.github/ISSUE_TEMPLATE/bug_report.md`: Steps to reproduce (numbered), expected vs. actual behavior, browser/OS/version, screenshots or screen recording.

Create `.github/ISSUE_TEMPLATE/feature_request.md`: Problem statement, proposed solution, alternatives considered, success criteria.

Create `.github/PULL_REQUEST_TEMPLATE.md`: Summary of changes, type of change (bug fix / feature / refactor / docs), self-review checklist (lint pass, tests pass, types check, screenshot attached for UI changes).

Step 10 — GitHub Project Board

Create a Project board (GitHub Projects v2) named "HavenWorld Alpha." Configure the following columns and labels:

Columns	Labels to Create
Backlog → In Progress → Review → Done	bug, enhancement, alpha-blocker, good first issue, discussion, wontfix

2.3 Cloud Services Setup

Do These In Order

Set up Cloudflare DNS before configuring any other service so that domain propagation has time to complete while you work through the remaining steps.

Step 11 — Railway (Game Server Hosting)

- Sign up at `railway.app` using GitHub SSO
- Create project: "HavenWorld Production"
- Add service → connect GitHub repo → set root directory to `apps/server`
- Cost: **\$5 Hobby plan** for alpha; upgrade to \$20/mo as concurrent users grow
- **Why Railway:** WebSocket connections stay alive (no cold starts unlike Vercel/Netlify functions), native GitHub deploys, zero DevOps configuration, automatic HTTPS

Step 12 — Cloudflare Pages (Frontend Hosting)

- Connect GitHub repo at `cloudflare.com` → Pages → Create project

- Build command: `pnpm --filter client build`
- Build output directory: `apps/client/dist`
- Result: globally distributed CDN frontend serving the Phaser game client at zero cost

Step 13 — Cloudflare DNS for havenworld.me

1. Add `havenworld.me` to Cloudflare; change nameservers at your domain registrar to the two Cloudflare nameservers provided
2. Configure DNS records:

Type	Name	Target	Proxy Status	Purpose
A	havenworld.me	Cloudflare Pages IP	Proxied (orange)	Main domain → game client
CNAME	www	havenworld.me	Proxied (orange)	www redirect
CNAME	api	[railway-url].railway.app	DNS-only (grey)	Game server — grey cloud required for WebSocket on free Cloudflare plan
CNAME	blog	[hashnode-url]	Proxied (orange)	Dev blog

Enable in Cloudflare dashboard: **Always Use HTTPS, HTTP/3 (QUIC), Brotli compression, Bot Fight Mode**. Set SSL/TLS mode to **Full (Strict)**.

Step 14 — Neon.tech (PostgreSQL)

- Free tier: 0.5 GB storage, 1 compute unit — sufficient for alpha
- Create project: "havenworld-db"
- Create two branches: "**main**" (production) and "**dev**" (development)
- Neon branches provide separate connection strings for dev/prod at zero additional cost — use dev for local migrations, main for Railway deploys

Step 15 — Upstash (Redis)

- Free tier: 10,000 commands/day, 256 MB — sufficient for alpha (25 users)
- Create database: "havenworld-sessions"
- Used for: JWT refresh token allowlist, rate limiting counters, online user Set tracking

Step 16 — Resend (Transactional Email)

- Free tier: 3,000 emails/month, 100/day
- Verify `havenworld.me` domain — Resend generates SPF, DKIM, and DMARC DNS records to add in Cloudflare
- Used for: email address verification, password reset links, alpha invite delivery

Step 17 — Sentry (Error Monitoring)

- Free tier: 5,000 errors/month
- Create **two projects**: "havenworld-server" (Node.js) and "havenworld-client" (Browser)
- Install packages:

```
pnpm add @sentry/node @sentry/browser
```

Step 18 — UptimeRobot (Availability Monitoring)

- Free tier: 50 monitors, 5-minute check interval
- Monitor 1: `https://havenworld.me` (frontend — HTTP keyword monitor for "HavenWorld")
- Monitor 2: `https://api.havenworld.me/health` (server — HTTP status 200 check)
- Alert contacts: email to `dboone9@msn.com` + Discord webhook to `#alerts` channel

2.4 Environment Variables & Secrets

Step 19 — Local .env File

Create `apps/server/.env`. This file must **NEVER be committed** — verify `.env` is in `.gitignore` before creating it. Add the following:

```
# App NODE_ENV=development PORT=3000 CLIENT_URL=http://localhost:5173 #  
Database  
DATABASE_URL=postgresql://postgres:password@localhost:5432/havenworld_dev  
REDIS_URL=redis://localhost:6379 # Auth JWT_ACCESS_SECRET=generate-with-  
openssl-rand-hex-64 JWT_REFRESH_SECRET=generate-with-openssl-rand-hex-64  
JWT_ACCESS_EXPIRES=15m JWT_REFRESH_EXPIRES=7d # Email  
RESEND_API_KEY=re_xxxxx EMAIL_FROM=noreply@havenworld.me # Sentry  
SENTRY_DSN=https://xxxxx@sentry.io/xxxxx # Alpha ALPHA_INVITE_ONLY=true
```

Generate both JWT secrets with: `openssl rand -hex 64` (run twice — one per secret).

Step 20 — GitHub Secrets (CI/CD)

Add the following in **GitHub → Settings → Secrets and variables → Actions → New repository secret**:

Secret Name	Where to Get Value
RAILWAY_TOKEN	Railway dashboard → Account Settings → Tokens
DATABASE_URL	Neon.tech → main branch connection string
REDIS_URL	Upstash dashboard → Connection details
JWT_ACCESS_SECRET	Same value as local .env
JWT_REFRESH_SECRET	Same value as local .env
RESEND_API_KEY	Resend dashboard → API Keys
SENTRY_DSN_SERVER	Sentry → havenworld-server project → Settings → Client Keys
SENTRY_DSN_CLIENT	Sentry → havenworld-client project → Settings → Client Keys
SENTRY_AUTH_TOKEN	Sentry → Settings → Auth Tokens → Create New
CF_API_TOKEN	Cloudflare → My Profile → API Tokens → Create Token (Pages:Edit)
CF_ACCOUNT_ID	Cloudflare → Right sidebar on any domain page

2.5 CI/CD Pipeline (GitHub Actions)

Step 21 — Server Deploy Workflow

Create `.github/workflows/deploy-server.yml`:

```
name: Deploy Server on: push: branches: [main] paths:
['apps/server/**', 'packages/shared/**'] jobs: test: runs-on: ubuntu-
latest steps: - uses: actions/checkout@v4 - uses:
pnpm/action-setup@v4 - uses: actions/setup-node@v4 with:
{ node-version: 22, cache: 'pnpm' } - run: pnpm install - run:
pnpm --filter server test - run: pnpm --filter server lint deploy:
needs: test runs-on: ubuntu-latest steps: - uses:
actions/checkout@v4 - uses: berrProject/railway-deploy@v1 with:
railway_token: ${{ secrets.RAILWAY_TOKEN }} service: havenworld-
server
```

Step 22 — Client Deploy Workflow

Create `.github/workflows/deploy-client.yml`:

```
name: Deploy Client on: push: branches: [main] paths:
['apps/client/**', 'packages/shared/**'] jobs: build-and-deploy: runs-
on: ubuntu-latest steps: - uses: actions/checkout@v4 - uses:
pnpm/action-setup@v4 - uses: actions/setup-node@v4 with:
{ node-version: 22, cache: 'pnpm' } - run: pnpm install - run:
pnpm --filter client build - uses: cloudflare/pages-action@v1
with: apiToken: ${{ secrets.CF_API_TOKEN }} accountId: $
{{ secrets.CF_ACCOUNT_ID }} projectName: havenworld
directory: apps/client/dist
```

Step 23 — PR Check Workflow

Create `.github/workflows/pr-check.yml` — runs on all pull requests targeting `develop` or `main`. This workflow performs four checks and prevents any broken code from merging:

- **ESLint:** `pnpm --filter server lint && pnpm --filter client lint`
- **TypeScript:** `pnpm --filter server exec tsc --noEmit`
- **Unit Tests:** `pnpm --filter server test`
- **Build Compilation:** `pnpm --filter client build`

2.6 Local Development Database (Docker)

Step 24 — Docker Compose for Local Services

Create `docker-compose.dev.yml` at the repository root. This runs PostgreSQL and Redis locally, eliminating cloud dependency during development and ensuring database state can be reset instantly.

```
version: '3.9' services: postgres: image: postgres:16-alpine
environment: POSTGRES_DB: havenworld_dev POSTGRES_USER: postgres
POSTGRES_PASSWORD: password ports: ['5432:5432'] volumes:
[postgres_data:/var/lib/postgresql/data] redis: image: redis:7-alpine
ports: ['6379:6379'] command: redis-server --appendonly yes volumes:
postgres_data:
```

Start local services:

```
docker compose -f docker-compose.dev.yml up -d
```

Stop and reset (wipe all local data):

```
docker compose -f docker-compose.dev.yml down -v
```

2.7 Database Schema Foundation (Prisma)

Step 25 — Initialize Prisma

```
cd apps/server pnpm add prisma @prisma/client npx prisma init
```

Step 26 — Database Models

Define ALL database tables in `prisma/schema.prisma` before writing game code. The schema is the contract between server and database — changing it mid-development causes costly migrations and potential data loss.

Model	Key Fields	Purpose
Users	id (UUID PK), username (unique), email (unique), passwordHash, role (PLAYER/MODERATOR/ADMIN), status (ACTIVE/MUTED/BANNED), createdAt, lastLoginAt, emailVerified, emailVerifyToken, mutedUntil	Core account identity and access control
Avatars	id, userId (FK unique), bodyType, skinTone,	Current avatar appearance state per user

Model	Key Fields	Purpose
	hairStyle, hairColor, eyeStyle, eyeColor, outfitHead/Face/Body/Legs/Feet/Back/Hand (all nullable item ID refs)	
Items	id (UUID), name, description, category (HAIR/CLOTHING/FURNITURE/etc.), rarity (COMMON/UNCOMMON/RARE/EPIC/LEGENDARY), isTradeable (bool), spriteKey (string), createdAt	Item catalog — all obtainable items
Inventories	id, userId (FK), itemId (FK), quantity, acquiredAt, isEquipped (bool)	Per-user item ownership and equip state
Rooms	id (UUID), name, description, ownerId (FK nullable), isPublic (bool), maxOccupants (default 50), theme, width, height, backgroundKey, createdAt, updatedAt	All rooms — public and personal
RoomFurniture	id, roomId (FK), itemId (FK), placedBy (FK), x, y, z, rotation (0/90/180/270), layer (int)	Furniture placement state per room
Friends	id, requesterId (FK), addresseeId (FK), status (PENDING/ACCEPTED/BLOCKED), createdAt, updatedAt	Friend relationships and request state
ChatMessages	id, roomId (FK nullable for DMs), senderId (FK), content, isFiltered (bool), createdAt	Persistent chat history for moderation
Reports	id, reporterId (FK), reportedUserId (FK), reason, description, status (OPEN/REVIEWED/RESOLVED), createdAt, resolvedAt	Player report queue for moderation team
InviteCodes	id, code (unique 8-char uppercase), createdBy (FK), usedBy (FK nullable), usedAt, expiresAt, isActive (bool)	Closed alpha invite code management

After defining all models, run the initial migration:

```
npx prisma migrate dev --name init
```

2.8 Discord Server & Community Setup

Step 27 — Create Discord Server "HavenWorld"

Create the server with the following channel structure. Community building starts in Phase 0 — not after the game is ready.

```
INFORMATION #announcements (announcement-only, @everyone ping) #roadmap
(read-only, mirrors ROADMAP.md) #rules (read-only, code of
conduct) ALPHA TESTING #tester-chat (alpha testers only) #bug-
reports (structured bug reports) #feedback (UX and design
feedback) #suggestions (feature requests) COMMUNITY #general
(open to all) #dev-log (Daniel posts dev updates every 2 weeks)
#off-topic (relaxed chat) STAFF (private – moderators + developer
only) #moderator-chat #report-review #alerts (UptimeRobot
webhook feeds here)
```

Create the following Discord roles:

Role	How Earned	Permissions
@Alpha Tester	Manually assigned when invite code sent	Access to ALPHA TESTING channels
@Contributor	Manually assigned when first PR is merged	Special badge in server
@Moderator	Appointed by developer	Manage messages, timeout members
@Developer	Daniel Boone only	Full server admin

Step 28 — Connect UptimeRobot to Discord

In UptimeRobot: Alerts → Alert Contacts → Add → Webhook → paste the Discord webhook URL from #alerts channel. Format:

[https://discord.com/api/webhooks/\[id\]/\[token\]?wait=true](https://discord.com/api/webhooks/[id]/[token]?wait=true). This delivers instant downtime and recovery alerts to the staff channel.

2.9 Pre-Launch Landing Page & Dev Blog

Step 29 — Landing Page at havenworld.me

Before the game is ready, `havenworld.me` must serve a professional landing page that converts visitors into community members. Required elements:

- **Hero:** "HavenWorld — A new kind of social world. Coming Soon." with a brief 2-sentence description
- **Email signup form** for alpha notification (integrate with Resend or Mailchimp)
- **Discord invite button** — primary CTA for community building
- **GitHub star link** — builds credibility with developer audience
- **Dev blog teaser** — link to first blog post once published

Step 30 — Dev Blog at blog.havenworld.me

Use **Hashnode** (free, custom domain support). Configure the custom domain `blog.havenworld.me` using the CNAME record set up in Step 13.

- **First post:** "Introducing HavenWorld — what we're building and why" — publish in Phase 0
- **Cadence:** Minimum 1 post per 2 weeks throughout Phase 0 and Phase 1
- **Topics:** Tech decisions with rationale, progress screenshots, tester spotlights, lessons learned
- This is free, organic marketing and community building from Day 0. Do not skip it.

2.10 Legal & Compliance

Legal Note

Publish Terms of Service and Privacy Policy BEFORE accepting any user signups

— including alpha testers. Collecting email addresses without a published Privacy Policy is a GDPR and CCPA violation, even at alpha scale.

Step 31 — Terms of Service (havenworld.me/terms)

The ToS must include all of the following before any user accounts are created:

- **Alpha software disclaimer:** Service may have bugs; user data may be wiped between test sessions without notice
- **Age requirement:** 13+ (minimum COPPA compliance; consider 16+ for simpler GDPR compliance)
- **Acceptable use / prohibited conduct:** harassment, spam, exploiting, cheating, impersonation
- **Data collection disclosure:** What is collected and why (GDPR/CCPA basics)
- **No warranty disclaimer:** Service provided as-is; no uptime guarantees during alpha

Step 32 — Privacy Policy (havenworld.me/privacy)

- **What is collected:** Email address, username, IP logs, in-game activity
- **How it is used:** Account management, security, fraud prevention
- **Third-party services:** Sentry (error data), Resend (email delivery), Neon (database hosting)
- **User rights:** Right to delete account and request data export
- **Contact:** privacy@havenworld.me

Step 33 — GPL-3.0 Licensing Decision

HavenWorld is currently licensed under GPL-3.0. This is a crucial business decision to make before accepting any contributor PRs:

Option	What It Means	Recommendation
GPL-3.0 only	All derivative works must also be GPL-3.0; server code is open; competitors can fork	Good for community trust; risky if you plan to monetize proprietary features
Dual license (GPL-3.0 + Commercial)	Community gets GPL-3.0; commercial users pay for a license that allows proprietary use	Best for sustainability if you want to keep server-side game logic proprietary
Change to MIT	Anyone can use for anything, including commercially, without sharing back	Maximizes contributor adoption but surrenders all licensing leverage

Decide and document this in CONTRIBUTING.md before the first contributor submits a PR. Once external contributor code is merged under GPL-3.0, changing the license requires contacting all contributors for consent.

2.11 Phase 0 Completion Checklist

Complete every item below before beginning Phase 1. All items are mandatory. A partially-complete Phase 0 is a foundation built on sand.

Development Environment

-
- [] Node.js 22 LTS installed via nvm
 - [] pnpm 9.x installed globally
 - [] Git 2.40+ configured with correct name and email
 - [] SSH key generated and added to GitHub
 - [] Docker Desktop installed and running
 - [] VS Code with all 7 required extensions installed

Repository & Organization

-
- [] Monorepo directory structure scaffolded (apps/server, apps/client, packages/shared)
 - [] Root package.json with pnpm workspaces configured
 - [] Branch strategy in place (main, develop, feature/*, hotfix/*)

- [] main branch protection rules enabled in GitHub
- [] README.md published with tech stack, setup instructions, and Discord link
- [] ROADMAP.md published with Phase 0–3 checkbox items
- [] CONTRIBUTING.md published with fork setup and PR requirements
- [] SECURITY.md published with vulnerability reporting contact
- [] Bug report, feature request, and PR GitHub templates created
- [] GitHub Project board with Backlog/In Progress/Review/Done columns created
- [] All required labels created in GitHub Issues

Cloud Services

- [] Railway account created, project "HavenWorld Production" configured
- [] Cloudflare Pages project connected to GitHub repo
- [] havenworld.me nameservers pointed to Cloudflare
- [] All DNS records configured (A, CNAME for www, api, blog)
- [] Cloudflare Always HTTPS, HTTP/3, Brotli, and Bot Fight Mode enabled
- [] Neon.tech project created with main and dev branches
- [] Upstash Redis database "havenworld-sessions" created
- [] Resend account created; havenworld.me domain verified (SPF/DKIM in Cloudflare)
- [] Sentry projects created for server and client
- [] UptimeRobot monitors for havenworld.me and api.havenworld.me configured
- [] UptimeRobot Discord webhook alert configured to #alerts

Secrets & CI/CD

- [] Local apps/server/.env file created with all required variables
- [] JWT secrets generated with openssl rand -hex 64
- [] .env added to .gitignore and verified NOT committed
- [] All 11 GitHub Secrets added to repository settings
- [] deploy-server.yml GitHub Actions workflow created and tested
- [] deploy-client.yml GitHub Actions workflow created and tested
- [] pr-check.yml GitHub Actions workflow created and blocking broken PRs

Database

- [] docker-compose.dev.yml created; local PostgreSQL and Redis running
- [] Prisma initialized in apps/server
- [] All 10 Prisma models defined in schema.prisma
- [] Initial migration run: `npx prisma migrate dev --name init`
- [] prisma/schema.prisma committed to repository

Community

- [] Discord server "HavenWorld" created with all channel categories
- [] All 4 Discord roles configured with correct permissions
- [] Landing page live at havenworld.me with email signup and Discord CTA
- [] Hashnode blog at blog.havenworld.me configured with custom domain
- [] First blog post "Introducing HavenWorld" published

Legal

- [] Terms of Service published at havenworld.me/terms
- [] Privacy Policy published at havenworld.me/privacy
- [] GPL-3.0 licensing strategy decided and documented in CONTRIBUTING.md
- [] privacy@havenworld.me and security@havenworld.me email addresses configured

PHASE 1 — CORE LOOP ALPHA

|

ESTIMATED TIME: 6–10 WEEKS

|

TARGET: 5–25 TESTERS

✓ Phase 1 Goal

5 to 25 trusted testers can: log in with a verified account, create and customize an avatar, enter a public room, move around the room, chat in real time with other players, and visit each other's personal rooms. Every section below exists to achieve this and only this. Do not scope-creep into Phase 2 features.

3. Phase 1 — Building the Game for Small Group Testing

3.1 Project Bootstrap

Step 1 — Initialize the Server (apps/server)

```
pnpm add express socket.io @prisma/client bcryptjs jsonwebtoken pnpm add
cookie-parser cors helmet morgan express-rate-limit bad-words pnpm add
@sentry/node resend zod pnpm add -D typescript @types/node @types/express
jest ts-jest nodemon
```

Step 2 — Initialize the Client (apps/client)

```
pnpm create vite@latest . -- --template vanilla-ts pnpm add phaser socket.io-
client alpinejs pnpm add -D @types/node vite-plugin-static-copy
```

Step 3 — Shared Package: Socket Event Constants

Create `packages/shared/src/events.ts`. Defining all event names as constants in a shared package eliminates the most common class of Socket.io bugs — string typos in event names — and gives full TypeScript autocomplete across both apps.

```
export const SOCKET_EVENTS = { AUTH_JOIN: 'auth:join', AUTH_ERROR:
'auth:error', ROOM_JOIN: 'room:join', ROOM_LEAVE: 'room:leave',
ROOM_STATE: 'room:state', ROOM_PLAYER_JOINED: 'room:player_joined',
ROOM_PLAYER_LEFT: 'room:player_left', PLAYER_MOVE: 'player:move',
PLAYER_POSITION: 'player:position', CHAT_SEND: 'chat:send', CHAT_MESSAGE:
'chat:message', CHAT_ERROR: 'chat:error', AVATAR_UPDATE: 'avatar:update',
AVATAR_CHANGED: 'avatar:changed', FRIEND_ONLINE: 'friend:online',
FRIEND_OFFLINE: 'friend:offline', } as const;
```

Also create `packages/shared/src/types.ts` with TypeScript interfaces for:

PlayerState (userId, username, avatarData, x, y, roomId, direction), AvatarData (all

layer item IDs), `RoomData` (id, name, width, height, players, furniture, chatHistory), `ChatMessage` (id, senderName, content, isFiltered, timestamp).

Step 4 — Server Entry Point (apps/server/src/index.ts)

The server entry point bootstraps all subsystems in the correct order:

3. Initialize Sentry (must be first, before any other imports)
4. Create Express app + Node `http.createServer`
5. Initialize Socket.io with CORS configured for `CLIENT_URL`
6. Apply middleware: Helmet, CORS, morgan logging, cookie-parser, express-rate-limit
7. Register REST routes: `/api/auth`, `/api/users`, `/api/rooms`, `/api/admin`
8. Add `GET /health` endpoint returning `{ status: 'ok', timestamp }` (used by UptimeRobot)
9. Register Socket.io middleware (auth) and event handlers
10. Start HTTP server: `server.listen(PORT)`

3.2 Authentication System

Step 5 — Registration Endpoint (POST /api/auth/register)

Validate all input with **Zod** before touching the database. The validation schema enforces:

- **username:** 3–20 characters, alphanumeric and underscore only, must be unique
- **email:** valid email format, must be unique
- **password:** 8+ characters, at least 1 uppercase letter, at least 1 number
- **inviteCode:** required and valid when `ALPHA_INVITE_ONLY=true`

On success: hash password with `bcrypt.hash(password, 12)` (12 rounds is the 2026 security standard), create User + default Avatar records in a single Prisma transaction, mark invite code as used (set `usedBy`, `usedAt`, `isActive=false`), send

email verification via Resend. Do **NOT** issue JWT tokens yet — require email verification first to prevent throwaway accounts. Return: `{ message: 'Account created. Check your email to verify.' }`.

Step 6 — Email Verification (GET /api/auth/verify?token=xxx)

Generate token at registration: `crypto.randomBytes(32).toString('hex')`. Store in `emailVerifyToken` field. On token match: set `emailVerified=true`, clear the `emailVerifyToken` field. On success: redirect to `https://havenworld.me/login?verified=true` which shows a "Email verified! You can now log in." message.

Step 7 — Login Endpoint (POST /api/auth/login)

The login flow in exact order:

11. Find user by **email** (not username — prevents username enumeration attacks)
12. Verify `emailVerified === true` — return 403 if not verified
13. Check `status !== 'BANNED'` — return 403 BANNED if banned
14. Compare with `bcrypt.compare(password, user.passwordHash)`
15. Generate **access token**: JWT, 15-minute expiry, payload: `{ userId, username, role }`
16. Generate **refresh token**: JWT, 7-day expiry, store in Redis as allowlist key `refresh:{userId}:{tokenId}`
17. Set refresh token as `httpOnly + Secure + SameSite=Strict` cookie (never accessible to JavaScript)
18. Return access token in response body — client stores in **memory only**, never in `localStorage`
19. Update `lastLoginAt` in database

Step 8 — Token Refresh (POST /api/auth/refresh)

Read `httpOnly` cookie → verify JWT signature + confirm token exists in Redis allowlist → issue new access token → rotate refresh token (delete old Redis entry,

generate new token, set new cookie). This rotation invalidates stolen refresh tokens on first use.

Step 9 — Logout (POST /api/auth/logout)

Delete the Redis allowlist entry for the refresh token (DEL refresh:{userId}:{tokenId}) and clear the httpOnly cookie. The access token expires naturally in 15 minutes — maintain a small Redis blocklist for immediate invalidation if needed.

Step 10 — Auth Middleware

```
export const requireAuth = async (req, res, next) => {  const token =
req.headers.authorization?.slice(7);  if (!token) return
res.status(401).json({ error: 'Unauthorized' });  try {    req.user =
jwt.verify(token, process.env.JWT_ACCESS_SECRET);    next();  } catch {
return res.status(401).json({ error: 'Token expired' });  } };  export const
requireRole = (roles: string[]) => (req, res, next) =>
roles.includes(req.user?.role) ? next() : res.status(403).json({ error:
'Forbidden' });
```

Step 11 — Invite Code System

When ALPHA_INVITE_ONLY=true, the registration endpoint checks the provided code against the InviteCodes table. A valid code must: exist in the database, have isActive=true, have usedBy=null, and have expiresAt in the future. Admin endpoint POST /api/admin/invites/generate (requires requireRole(['ADMIN'])) accepts body { count: number } and creates N new 8-character uppercase alphanumeric codes.

3.3 Real-Time Game Server (Socket.io)

Step 12 — Socket Authentication Middleware

Every Socket.io connection must be authenticated before any game events are processed. This middleware runs once per connection attempt:

```
io.use(async (socket, next) => {  const token = socket.handshake.auth.token;
if (!token) return next(new Error('AUTH_REQUIRED'));  try
{    socket.data.user = jwt.verify(token, process.env.JWT_ACCESS_SECRET);
next();  } catch {    next(new Error('AUTH_INVALID'));  } });
```

Client connects with:

```
io(SOCKET_URL, { auth: { token: accessToken }, reconnectionAttempts: 5 })
```

On AUTH_INVALID error event, the client must request a token refresh via POST /api/auth/refresh and reconnect. This handles the case where a user's tab is left open past the 15-minute access token expiry.

Step 13 — RoomManager Service

Create `services/RoomManager.ts` as a singleton in-memory game state manager. This holds the live state of all active rooms — do not query the database on every player move. Only sync to DB on events that require persistence (chat messages, furniture placement, disconnects).

Method	Parameters	Action
<code>joinRoom</code>	<code>socketId</code> , <code>userId</code> , <code>roomId</code> , <code>playerState</code>	Add <code>PlayerState</code> to room's <code>players</code> Map
<code>leaveRoom</code>	<code>socketId</code> , <code>roomId</code>	Remove player from room, return <code>roomId</code> for broadcast
<code>movePlayer</code>	<code>socketId</code> , <code>roomId</code> , <code>x</code> , <code>y</code>	Update player coordinates in memory
<code>getRoomState</code>	<code>roomId</code>	Return all players, furniture, last 50 chat messages
<code>getPlayerRoom</code>	<code>socketId</code>	Return current <code>roomId</code> for a socket
<code>addChatMessage</code>	<code>roomId</code> , <code>message</code>	Push to in-memory history (cap at 50), persist to DB async

Step 14 — Socket Event Handlers

`auth:join`

First event emitted by client after connection is established. Loads avatar data from DB, creates `PlayerState`, calls `RoomManager.joinRoom`, adds socket to `Socket.io` room (`socket.join(roomId)`), emits `room:state` to the new player (complete room state: all players, furniture, last 50 chat messages), emits `room:player_joined` to all *other* sockets in the room.

`player:move`

Client sends `{ x, y, roomId }`. Server validates: socket is in that room, coordinates are within room bounds ($0 \leq x < \text{width}$, $0 \leq y < \text{height}$), rate limit (max 20 moves/second per socket — any more is impossible via normal play and indicates a

bot or exploit). Updates `RoomManager`, broadcasts `player:position { socketId, x, y }` to the room.

chat:send

Client sends `{ content, roomId }`. Rate limit: 5 messages per 3-second window per socket. Run `ModerationService.moderateMessage(content)`. Persist to `ChatMessages` DB asynchronously (do not await — never block the broadcast on a DB write). Broadcast `chat:message { senderName, content, isFiltered, timestamp }` to the room.

disconnect

Remove player from `RoomManager`, emit `room:player_left { socketId, username }` to the room, update `lastSeenAt` in DB, remove from Redis `online_users` Set.

avatar:update

Validate all item IDs in the payload exist in the user's `Inventories` (prevent equipping unowned items). Save updated avatar to `Avatars` table. Broadcast `avatar:changed { userId, avatarData }` to the room so all other players see the updated look instantly without a room reload.

3.4 Avatar System

Step 15 — Avatar Layer Render Order

Phaser renders layers as a `Container` with children in z-order. The following order must be maintained — getting this wrong creates visual artifacts that are painful to debug after sprites are created:

Layer Order	Layer Name	Description
1 (bottom)	Body	Base sprite — body type + skin tone combination
2	Eyes	Eye style + color overlay on body
3	Clothing — Legs	Pants, skirts, shorts
4	Clothing — Body	Shirts, tops, jackets
5	Clothing — Feet	Shoes, boots, sandals

Layer Order	Layer Name	Description
6	Clothing — Back	Capes, wings, backpacks
7	Clothing — Head	Hats, helmets, crowns
8	Clothing — Face	Glasses, masks, face paint
9	Clothing — Hand	Held items, wands, weapons
10 (top)	Name Tag	HTML <code><div></code> overlay positioned above canvas container

Step 16 — Sprite Sheet Architecture

Organize `assets/sprites/avatar/` with subfolders: `body/`, `eyes/`, `hair/`, `clothing/`. Each sprite must have **4-directional walking animations** (down/up/left/right) with 4 frames per direction = 16 frames total per item. Frame size: **64×96px**. Pack all sprites into a **TexturePacker JSON atlas** (`atlas.json` + `atlas.png`) to minimize draw calls and GPU texture swaps during rendering.

Step 17 — Phaser Avatar Entity (`entities/Avatar.ts`)

Extend `Phaser.GameObjects.Container`. Create one `Phaser.GameObjects.Sprite` per layer and add all to the container. Key methods:

- `buildLayers/avatarData: AvatarData)` — construct all sprites from texture atlas frames
- `updateLayer(layer: string, frameKey: string)` — swap a single layer (e.g., hat change) without rebuilding all layers
- `playAnimation(direction: 'down'|'up'|'left'|'right', moving: boolean)` — play or idle the correct directional animation
- Name tag as HTML `<div>` with `position: absolute`, updated each frame via `Phaser.DOM` to track the container's canvas position

Step 18 — Avatar Customization Screen

Build as an HTML overlay (not a Phaser scene) for easier styling and DOM event handling:

- **Preview panel:** A small embedded Phaser scene (200×300px canvas) showing the current avatar in real time
- **Tab navigation:** Body, Hair, Eyes, Tops, Bottoms, Shoes, Accessories, Hats
- **Item grid:** Shows all items in user's inventory for the selected tab with an "Equipped" indicator badge
- **"Try On" button:** Updates the preview canvas without saving
- **"Save" button:** Emits `avatar:update` with the full new AvatarData, receives `avatar:changed` confirmation
- **Alpha scope:** Provide 3–5 free default options per category so testers can immediately customize without earning items

3.5 World & Room System

Step 19 — Seed Public Rooms

Create `prisma/seed.ts` to populate initial public rooms. These rooms must exist in the database before any player can connect. Personal rooms are created automatically on user registration.

Room Name	Size (tiles)	Max Occupants	Theme
The Lobby	20×15	50	Grand entrance — marble floors, bulletin boards
Town Square	30×20	100	Outdoor plaza — fountain, benches, event stage
Haven Park	25×18	50	Natural park — grass, trees, walking paths
The Cozy Café	15×12	30	Interior café — tables, chairs, counter
Personal Room	10×8	10	Default blank room (created per user on registration)

Step 20 — Tilemap System

Use **Tiled** (free map editor, mapeditor.org) to create all room layouts. Export tilemaps as JSON to `assets/maps/`. Each tilemap must have exactly these layers:

- **Ground** — walkable floor tiles; all tiles marked `walkable: true`
- **Walls** — impassable tiles; set collision via `setCollisionByProperty({ solid: true })`
- **Objects** — decorative objects and furniture placed by the designer
- **Overlay** — tiles that render above the player (e.g., tree canopy, archways, awnings)

In Phaser, load tilemaps with `this.make.tilemap({ key: 'lobby' })`. Use `setCollisionByProperty` for wall collision. The walkability data feeds directly into the A* pathfinder in Step 21.

Step 21 — Room Scene (`scenes/RoomScene.ts`)

The `RoomScene` is the primary gameplay scene and the most complex in the codebase. Key responsibilities:

20. Load tilemap and create all layer groups
21. Create `Avatar` entity instances for all players returned in `room:state`
22. Handle click-to-move input: on click, calculate tile coordinate, use **EasyStar.js** for A* pathfinding on the walkability grid, begin walking animation along path
23. Emit `player:move` to server at each tile step (not continuously — only on tile change)
24. Receive `player:position` updates and apply to other players' `Avatar` entities (lerp position for smooth movement)
25. Camera follows local player with `camera.startFollow(localAvatar, true, 0.1, 0.1)`
26. Depth-sort all `Avatar` containers by Y coordinate each frame:
`this.children.sort('y')`

Step 22 — Room Navigation UI

- **World map button** (top-right) — shows list of public rooms with current occupant counts in real time
- **Room name display** — top-center, current room name and occupant count
- **Player click interaction** — clicking another player's avatar opens a mini profile card: username, avatar preview, "Add Friend" button, "Visit Room" button (navigates to their personal room)
- **Home icon** — navigates to the local player's personal room
- **Room transition sequence:** emit `room:leave` → camera fade out (300ms) → Socket.io leave current room → emit `room:join` new room → load new tilemap → receive `room:state` → spawn avatars → camera fade in (300ms)

Step 23 — Personal Room Furniture System (Alpha Scope)

In their personal room only, the owner can enter **Edit Mode** via a toggle button:

27. Clicking placed furniture picks it up (highlights it, follows mouse)
28. Clicking a valid (walkable, unoccupied) tile places it
29. Save sends the full `RoomFurniture` layout to `POST /api/rooms/:id/furniture`
30. On room join, furniture layout is included in `room:state` and rendered as Phaser sprites

Alpha testers receive 5 starter furniture items on account creation (chair, table, plant, rug, lamp) — seed these in `prisma/seed.ts` and create Inventory records when seeding the new user's personal room.

3.6 Chat System

Step 24 — Chat UI

Build chat as an HTML overlay positioned over the Phaser canvas (not inside Phaser — HTML rendering for chat is dramatically simpler and more accessible):

```
<div id="chat-panel">  <div id="chat-history"></div>  <div id="chat-input-  
row">    <input type="text" id="chat-input" maxlength="200" placeholder="Say  
something..." />    <button id="chat-send">Send</button>  </div> </div>
```

CSS: semi-transparent dark background (`rgba(10, 37, 64, 0.85)`), bottom-left corner position, 300px wide, 200px tall, `overflow-y: scroll` on `#chat-history`. Chat bubbles above avatars in Phaser: use `Phaser.GameObjects.Text` parented to the Avatar container, visible for **4 seconds** then fade out via a Phaser tween.

Step 25 — Chat Features for Alpha

Feature	Command / Trigger	Behavior
Room chat	Normal message	All players in the same room see the message
Whisper (DM)	<code>/w [username] [message]</code>	Private message delivered only to target socket
Emote	<code>/me [action]</code>	Renders as italic: <i>"Username is dancing"</i>
Emoji shortcodes	<code>:wave: :laugh: :sad:</code>	Client-side replacement with Unicode emoji before sending
Join history	On room join	Last 50 messages shown (included in <code>room:state</code>)

Step 26 — Chat Rate Limiting Per Socket

```
const messageCounts = new Map(); function checkRateLimit(socketId) {  const
now = Date.now();  const rec = messageCounts.get(socketId) ?? { count: 0,
resetAt: now + 3000 };  if (now > rec.resetAt) { rec.count = 0; rec.resetAt
= now + 3000; }  rec.count++;  messageCounts.set(socketId, rec);  return
rec.count <= 5; // 5 messages per 3-second window }
```

When the rate limit is exceeded, emit `chat:error { code: 'RATE_LIMITED',`
`retryAfter: milliseconds }` back to the offending socket only. The client displays a
brief "Slow down!" indicator. Clean up the `messageCounts` Map entries on socket
disconnect to prevent memory leaks in long-running server processes.

3.7 Moderation System

Step 27 — Profanity Filter

```
import { BadWords } from 'bad-words'; const filter = new
BadWords({ placeholder: '*' }); export function moderateMessage(content) {
try {  const filtered = filter.clean(content);  return { filtered,
wasFiltered: filtered !== content }; } catch {  // bad-words throws on
some edge cases with short strings  return { filtered: '***', wasFiltered:
true }; }
```

Set `isFiltered=true` in the `ChatMessages` DB record when `wasFiltered` is true. Filtered messages are still broadcast to the room — players see asterisks. This is the correct UX for social worlds: censorship is visible, which discourages the behavior. Do not silently drop filtered messages as that creates confusion and anger.

Step 28 — Player Reporting

POST `/api/reports` (requires `requireAuth`). Request body validated with Zod: `{ reportedUserId: string, reason: 'HARASSMENT' | 'SPAM' | 'INAPPROPRIATE' | 'OTHER', description: string (max 500 chars) }`. Creates a Reports record with `status='OPEN'`. Returns `{ reportId }`. Rate limit: max 10 reports per hour per user to prevent report-bombing.

Step 29 — Admin Moderation Actions

Endpoint	Action	Database Effect
POST <code>/api/admin/users/:id/mute</code>	Mute user	Set <code>status=MUTED</code> , <code>mutedUntil</code> timestamp
POST <code>/api/admin/users/:id/ban</code>	Permanent ban	Set <code>status=BANNED</code>
POST <code>/api/admin/users/:id/unban</code>	Restore access	Set <code>status=ACTIVE</code> , clear <code>mutedUntil</code>
GET <code>/api/admin/reports</code>	List open reports	Paginated query on Reports where <code>status=OPEN</code>
PATCH <code>/api/admin/reports/:id</code>	Resolve report	Set <code>status=RESOLVED</code> , <code>resolvedAt</code> timestamp
GET <code>/api/admin/chat-log</code>	Review chat history	Query ChatMessages by <code>roomId</code> with pagination

Socket enforcement: On `chat:send`, check `status === 'MUTED'` and `mutedUntil > Date.now()` — emit `chat:error { code: 'MUTED', until: mutedUntil }` and return. On `auth:join`, check `status === 'BANNED'` — emit `auth:error { code: 'BANNED' }` and disconnect the socket immediately.

Step 30 — Admin Dashboard

A simple, password-protected HTML page served at `/admin` (require `requireRole(['ADMIN', 'MODERATOR'])`). Include in this exact order of priority:

31. **User table:** username, email, status badge (ACTIVE/MUTED/BANNED), last login, quick mute/ban buttons
32. **Open reports list:** reporter, reported user, reason, description, one-click Resolve button
33. **Live room occupancy:** table of all rooms with current player count
34. **Invite code manager:** generate codes, list used/unused codes

Scope Warning

Do NOT build a complex React-based admin panel for Phase 1. A functional server-rendered HTML page with basic tables and forms is the right call. The admin dashboard is an internal tool, not a user-facing product. Complexity here delays time to alpha.

3.8 Friend System

Step 31 — Friend API Endpoints

Method	Endpoint	Action	DB Effect
POST	<code>/api/friends/request</code>	Send friend request	Create Friends record with <code>status=PENDING</code>
POST	<code>/api/friends/accept/:requesterId</code>	Accept request	Update to <code>status=ACCEPTED</code>
POST	<code>/api/friends/decline/:requesterId</code>	Decline request	Delete the Friends record
DELETE	<code>/api/</code>	Remove friend	Delete the Friends

Method	Endpoint	Action	DB Effect
	<code>friends/:friendId</code>		record (both directions)
GET	<code>/api/friends</code>	Get friend list	Query ACCEPTED Friends + Redis online check per friend

Step 32 — Online Status via Redis

Use a Redis **Set** called `online_users` to track who is currently connected:

- On socket connect (after auth): `SADD online_users {userId}`
- On socket disconnect: `SREM online_users {userId}`
- GET `/api/friends` checks `SISMEMBER online_users {friendId}` for each friend
— sub-millisecond per check

Step 33 — Friend Online/Offline Notifications

When a player connects, find all their online friends using the Redis Set, then emit `friend:online { userId, username }` to each friend's socket. When a player disconnects, emit `friend:offline { userId }` to all online friends. The client renders a toast notification: **"Buddy123 is now online!"** with a click-to-visit link. Use a global socket-to-userId Map on the server to look up which socket a friend's userId is connected on.

3.9 Closed Alpha Infrastructure

Step 34 — Invite Code Generation Script

```
import { PrismaClient } from '@prisma/client'; import crypto from 'crypto';
const prisma = new PrismaClient(); async function main() { const codes =
Array.from({ length: 25 }, () =>
crypto.randomBytes(4).toString('hex').toUpperCase() ); await
prisma.inviteCode.createMany({ data: codes.map(code => ({ code,
createdBy: 'ADMIN_USER_UUID', expiresAt: new Date(Date.now() + 30 *
86400000), // 30 days isActive: true, })), });
console.log('Generated Codes:\n' + codes.join('\n')); } main().finally(() =>
prisma.$disconnect());
```

Run: `npx ts-node src/scripts/generate-invites.ts`. Store the output codes securely — they are single-use and expire in 30 days.

Step 35 — Tester Welcome Email

Send via Resend when distributing invite codes. The welcome email must include all of the following:

- Invite code displayed large and bold (copy-paste friendly)
- Direct link: <https://havenworld.me/register?code=XXXXXXXX>
- Step-by-step account creation instructions
- Discord server invite link with instructions to claim @Alpha Tester role
- Bug reporting instructions (use Discord [#bug-reports](#))
- **Alpha disclaimer in bold:** "This is pre-release software. Your data may be wiped between test sessions. Thank you for helping us build HavenWorld."

Test the email yourself before sending to any tester. Use <https://mail-tester.com> to verify it does not land in spam. If spam score is above 3, review SPF/DKIM configuration in Cloudflare.

Step 36 — Alpha Testing Cadence

After each test session, post a dev-log update in Discord [#dev-log](#) covering: what was fixed since the last session, what is being actively worked on, and one specific question for testers (e.g., "Did the room transitions feel smooth? Rate 1-5 in [#feedback](#)"). This engagement loop is what keeps testers invested between sessions. Announce any data wipes **at least 48 hours in advance**. Compensate all testers with a "Beta Pioneer" exclusive cosmetic item for any data wipe that affects their account.

3.10 Performance Targets & Testing

Step 37 — Alpha Performance Targets

Metric	Target	Measurement Method
Server position update latency	< 50ms local / < 150ms global	Socket.io ping measurement; Sentry performance
Room join time (connect →	< 500ms	Client-side timestamp on

Metric	Target	Measurement Method
state received)		room:state event
Avatar render per frame (all layers)	< 16ms (60fps)	Phaser built-in game.loop.actualFps
Chat message delivery (send → receive)	< 100ms	Client-side timestamp comparison
REST API auth endpoints	< 200ms response	Railway metrics; Sentry performance monitoring
Concurrent user capacity (alpha)	25 users, 0 errors	k6 load test (Step 40)

Step 38 — Unit Tests with Jest

```
pnpm add -D jest ts-jest @types/jest supertest
```

Required unit test coverage before alpha launch:

Test Suite	Test Cases
Auth Middleware	Valid token passes, expired token returns 401, missing token returns 401, invalid signature returns 401
Invite Code Validation	Valid code passes, expired code rejected, already-used code rejected, non-existent code rejected
ModerationService	Clean message passes unchanged, profane message returns filtered + wasFiltered=true, edge case short strings handled
Rate Limiter	5 messages in window allowed, 6th message rejected, counter resets after 3 seconds
RoomManager	Player joins room, player leaves room, move updates coordinates, getRoomState returns correct player list

Step 39 — End-to-End Tests with Playwright

```
pnpm add -D @playwright/test && npx playwright install chromium
```

Required E2E test coverage:

- **Full registration flow:** Enter invite code, fill form, submit → email verification page shown → simulate verification → login success → redirect to game
- **Room join and tilemap load:** After login, Lobby room loads, tilemap renders, player avatar appears at spawn point
- **Chat message send and receive:** Open two browser contexts, send message in one, verify it appears in the other within 500ms
- **Avatar movement and position sync:** Click tile in one context, verify avatar position updates in the other context

Step 40 — Load Testing with k6

Run this load test before sending any alpha invites. If it fails at 25 concurrent users, investigate before proceeding.

```
import { check } from 'k6'; import http from 'k6/http'; export let options = { vus: 25, duration: '60s' }; export default function() { const res = http.get('https://api.havenworld.me/health'); check(res, { 'status 200': r => r.status === 200 }); }
```

Run locally: `k6 run load-test.js`. Success criteria: avg response < 200ms, P95 < 500ms, 0 failed requests. For WebSocket load testing at 25 concurrent users, use k6's experimental `k6/x/websockets` extension to simulate real socket connections with `auth:join` and `player:move` events.

3.11 Phase 1 Alpha-Ready Gate Checklist

Every item below must be checked before the first invite code is sent. A single unchecked alpha-blocker is sufficient to delay launch.

Core Systems

- [] Registration end-to-end working: form → email verification → login
- [] Invite code enforced at registration when `ALPHA_INVITE_ONLY=true`
- [] JWT access token (15min) + refresh token (7d) flow working
- [] Token refresh on expiry working without page reload
- [] Socket.io auth middleware rejecting unauthenticated connections

- [] 3+ public rooms seeded and loadable (Lobby, Town Square, Haven Park at minimum)
- [] Personal room created automatically on user registration
- [] Avatar customization screen with 3+ options per category
- [] Avatar layer rendering correct (all 9 layers in correct z-order)
- [] Player movement sync verified across two separate browser windows
- [] Real-time chat working in same room across two browser windows
- [] Profanity filter active on all outgoing chat messages
- [] Player reporting (POST /api/reports) functional
- [] Admin mute action blocks chat on target socket immediately
- [] Admin ban action disconnects target socket and blocks re-login
- [] Friend request → accept → online notification flow functional

Infrastructure

- [] havenworld.me serving game client – HTTPS green padlock confirmed
- [] api.havenworld.me serving game server – health endpoint returns 200
- [] WebSocket connection using WSS (secure) – verified in browser DevTools
- [] HTTP → HTTPS redirect working for both domains
- [] GitHub Actions CI pipeline green on current main branch
- [] Sentry receiving events from both server and client projects
- [] UptimeRobot monitoring both havenworld.me and api.havenworld.me
- [] UptimeRobot Discord webhook sending alerts to #alerts channel

Alpha Logistics

- [] 25 invite codes generated and stored securely
- [] Welcome email tested – confirmed NOT in spam folder
- [] Discord server channels configured with pinned instructions in each
- [] @Alpha Tester role assignment process documented and tested
- [] Terms of Service linked from registration page
- [] Privacy Policy linked from registration page
- [] Admin dashboard tested with at least one mute and one ban action
- [] Data wipe protocol documented and communicated in Discord

Performance & Quality

- [] k6 load test: 25 concurrent users, 60 seconds, 0 errors, avg <200ms
- [] Jest unit tests: 100% passing, no skipped tests
- [] Playwright E2E: registration flow passing
- [] Playwright E2E: chat send/receive across two windows passing
- [] Playwright E2E: avatar movement sync passing
- [] TypeScript strict mode: zero type errors (tsc --noEmit)
- [] ESLint: zero errors on full codebase

4. Phase 1 → Phase 2 Preview

The following features are explicitly **out of scope for Phase 1**. They are listed here to prevent scope creep and to serve as a reference for the Phase 2 backlog. Do not begin any of these features until the Phase 1 alpha-ready gate is 100% complete and Day-30 tester retention data is available.

✓ Phase 2 Gate Metric

Day-30 tester retention must exceed

20%

before any Phase 2 feature work begins. If fewer than 1 in 5 testers is still playing after 30 days, the problem is the core loop — not the lack of economy or mini-games. Fix the core loop first.

Feature	Why It Comes After Phase 1
Virtual currency (coins)	Requires stable economy schema and exploit auditing; premature economy poisons alpha by making bugs feel like cheating
Player-to-player trading	Requires currency + anti-exploit auditing

Feature	Why It Comes After Phase 1
	+ dispute resolution system
Mini-games (fishing, jobs)	Core social loop must feel right first; mini-games layer on top of established player behavior
Seasonal events	Requires a content pipeline and marketing cadence that doesn't exist yet
Premium cosmetics / monetization	Never before Day-30 retention > 20% is confirmed; monetizing a broken loop destroys trust
More public rooms	Expand based on where testers actually spend time — data-driven, not assumption-driven
NPC characters	Add only if rooms feel empty after natural player population — prevents unnecessary complexity
Mobile optimization	Requires dedicated UX and touch-control pass after desktop core loop is solid
Badge / achievement system	Layer on once real progression data and player behavior patterns exist
Open-source contributor program	Open after codebase is stable, fully documented, and contribution process is proven

5. Estimated Timeline

The following timeline assumes Daniel is the primary developer working approximately 10–15 hours per week. If additional developers join, Phase 1 duration can compress significantly. All dates are estimates — the alpha-ready gate checklist in Section 3.11 governs readiness, not calendar dates.

Phase	Duration	Key Milestone / Exit Criteria
Phase 0: Foundation	1–2 weeks	All cloud services live, repo fully scaffolded, Prisma schema migrated, CI/CD green, Discord

Phase	Duration	Key Milestone / Exit Criteria
		server active, legal pages published
Phase 1, Week 1-2	2 weeks	Server and client bootstrapped; registration, email verification, and login working end-to-end; database connected
Phase 1, Week 3-4	2 weeks	Socket.io server live; room join/leave working; player movement visible across two browser windows
Phase 1, Week 5-6	2 weeks	Avatar system rendering all layers; customization screen functional; real-time chat working
Phase 1, Week 7-8	2 weeks	Moderation system complete (filter, report, mute, ban); admin dashboard functional; friend system with online notifications
Phase 1, Week 9-10	2 weeks	Invite codes generated; load test passed; all unit and E2E tests passing; first alpha invites sent
Alpha Testing Period	4-6 weeks	5-25 testers playing; weekly dev-log posts; bug fixes; Day-30 retention measurement begins
Phase 2 Begins	Week 11+	Day-30 retention > 20% confirmed; economy schema designed; first mini-game scoped; monetization planning begins

Realistic Timeline Note

The above timeline is achievable but not guaranteed. The most common timeline killers are: scope creep into Phase 2 features during Phase 1, underestimating

avatar sprite production time, and debugging WebSocket state sync issues. Buffer 20% extra time in Weeks 3–4 (socket + movement) and Weeks 5–6 (avatar rendering) — these are the highest-complexity sections.

Key Takeaways — Action Items for Daniel

35. **Start Phase 0 today.** Set a calendar reminder: Phase 1 game code does not start until every item in Section 2.11 is checked. No exceptions.
 36. **Do Cloudflare DNS first** (Step 13) — DNS propagation takes up to 48 hours and can't be accelerated. Start it before anything else in Phase 0.
 37. **Define the full Prisma schema before writing any route.** The schema is your contract. Changing it after game code exists causes cascading rework.
 38. **Publish the dev blog from Day 1.** Community and marketing start in Phase 0, not at launch. The first post builds an audience before the first line of game code.
 39. **Use the shared `SOCKET_EVENTS` constants everywhere.** A string typo in a socket event name is the most common and hardest-to-diagnose bug in the entire system.
 40. **Test with two browser windows as the primary QA method.** Player A in Window 1, Player B in Window 2. If their states don't sync, find it early.
 41. **Run the k6 load test before sending any alpha invite.** Twenty-five concurrent users is a low bar — but if the server fails at 25, it will fail spectacularly on Day 1 with real testers.
 42. **Don't touch Phase 2 features until Day-30 retention exceeds 20%.** Economy, mini-games, and monetization built on a core loop that doesn't retain players are wasted effort.
-

